

For each question, no points will be given without an explanation. There are three pages.

$$DB_1 =$$

Tid	Transaction
1	A B C
2	A B C F
3	A B F
4	B C E F
5	D E

1 Eclat (5 pts)

We use Eclat algorithm to compute all itemsets X with $\text{sup}(X) \geq 2$ in DB_1 . We assume the order on the items used by Eclat is alphabetical order $A, B, \dots$ meaning the first projected database is $DB_1[A]$.

You will not do a complete simulation of Eclat, but the following questions are about this execution.

Question 1: The Eclat algorithm uses a particular representation of databases. Give the representation of the database DB_1 used by Eclat.

Question 2: Later in the execution, Eclat computes $DB_1[A]$. Give its representation. How is it computed from the answer of the previous question?

Question 3: Which projected databases $DB_1[X]$ (where X is just one item) will be computed by Eclat and why?

Question 4: Give the representations of $DB_1[B]$ and $DB_1[BC]$.

Question 5: What are all the frequent itemsets in $DB_1[B]$? (Not just the frequent items. It is not necessary to simulate Eclat, just give the result). Explain what are the corresponding frequent itemsets in DB_1 and how they are computed.

2 Closed itemsets (5 pts)

Recall that:

- The support set $\text{sset}(X)$ of an itemset X is the set of all transactions that contain X ;
- two itemsets X and Y are equivalent if $\text{sset}(X) = \text{sset}(Y)$;
- a closed itemset is the largest itemset in an equivalence class. There is only one closed itemset in each equivalence class.
- the closure $\text{cl}(X)$ of X is the closed itemset in the equivalence class of X . It can be computed by taking the intersection of all transactions in $\text{sset}(X)$.

$$DB_2 =$$

	A	B	C	D
1	1	1	1	1
2	1	1	1	
3	1	1	1	1
4	1	1		
5				1

Lattice of itemsets :

					ABCD			
					ABC	ABD	ACD	BCD
					AB	AC	AD	BC
					A	B	C	D

It is possible to compute the closed itemset efficiently using a variation of the Apriori algorithm.

We define the C_{free} constraint as: $C_{\text{free}}(X)$ is true if and only if the support of X is different from the support of all its strict subsets. For instance $C_{\text{free}}(AB)$ is false because $\text{supp}(AB) = \text{supp}(A)$ and $C_{\text{free}}(CD)$ is true because $\text{supp}(CD) \neq \text{supp}(C)$; $\text{supp}(CD) \neq \text{supp}(D)$ and $\text{supp}(CD) \neq \text{supp}(\emptyset)$. (If n is the size of X , it is actually enough to check the supports of the subsets of size $n-1$ and not all of them).

The itemsets that satisfy C_{free} are called free itemsets. They are the minimal itemsets of each equivalence class (there can be several free itemsets in each equivalence class). Once the free itemsets are computed, the closed itemsets are the closure of the free itemsets.

It is possible to show that C_{free} is anti-monotonic. Thus we can compute efficiently all the free itemsets using a variation of the Apriori algorithm as follow. The differences with Apriori is that it use C_{free} instead of the minimum support constraint; instead of starting with itemsets of size 1, it starts with itemsets of size 0 and in line 4, it computes the closures of all free itemsets.

- $i = 0$
- $C_0 = \{\emptyset\}$
- while $C_i \neq \emptyset$ do
 - $F_i = \{X \in C_i \text{ such that } C_{\text{free}}(X) \text{ is true.}\}$
 - $C_{i+1} = \{X \text{ such that } |X| = i+1 \text{ and all } Y \subset X, \text{ with } |Y| = i \text{ are in } Y \in F_i\}$
 - $c_i = i+1$
- ClosedItemsets = $\{\text{cl}(X) \text{ for } X \in \cup_{0 \leq c_i < F_i\}$

C_i is the collection of candidate itemsets of size i , F_i is the collection of free itemsets of size i .

Question 6: Do a simulation of this algorithm on DB_2 . For each value of i , give the collections C_i and F_i and give ClosedItemsets at the end.

Question 7: Give the equivalence classes in DB_2 (you can draw them on the lattice of itemsets).

3 Krimp (4 pts)

The input of the Krimp algorithm are the database DB_3 and the set of candidates $F = \{AB, ABC, CD\}$. In the code tables below, only the size of the codes of the itemsets are given (in bits), not the codes themselves (since we are not interested in the codes). ST is the standard code table.

$$DB_3 =$$

Tid	Transaction
1	ABC
2	ABC
3	AB
4	CD
5	ABCD

$$CT_1 =$$

Itemset	Code Size
AB	1.3
C	1.3
D	2.3

$$ST =$$

Itemset	Code Size
A	1.8
B	1.8
C	1.8
D	2.8

Question 8: At the beginning of Krimp, the code table is ST. After the first iteration of Krimp, the code table is CT_1 . Compute $\text{Len}(DB_3 | CT_1)$ (the size of the database DB_3 when compressed using code table CT_1) and then $\text{TotLen}(DB_3, CT_1)$ assuming that the size of CT_1 (using ST) is $\text{Len}(CT_1) = 13.1$

Question 9: We denote CT_2 the code table built by Krimp in the 2nd iteration. Give CT_2 , compute the code sizes in CT_2 and the total size of CT_2 using the given standard code table ST.

Question 10: What is the code table at the end of this 2nd iteration of Krimp algorithm (and why)?

Use the logarithm table below.

Recall that $\log_2(ab) = \log_2(a) + \log_2(b)$ and thus $\log_2(a/b) = \log_2(a) - \log_2(b)$.

x	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
$\log_2(x)$	0	1	1.6	2	2.3	2.6	2.8	3	3.2	3.3	3.5	3.6	3.7	3.8	3.9	4

4 Krimp 2 (3 pts)

This exercise is independent and does not use the same database as the previous one. The goal is to simulate the krimp algorithm at a high level. The following table contains the length of the code table and the length of the database compressed using the code table. In the code table part, a 1 means that the itemset is in the code table and a 0 means it is not. The code tables always contain the items (i.e., itemsets of size 1) if it is necessary. The given lengths are computed as in krimp (after correctly ordering the itemsets in the code table and removing the itemsets with usage = 0).

For instance, the first row mean that the code table contain none of the itemsets AB, ABC and AC (it is therefore the standard code table ST).

AB	ABC	AC	Len(CT)	Len(DB CT)
0	0	0	7	31
0	0	1	10	22
0	1	0	9	25
0	1	1	12	14
1	0	0	9	22
1	0	1	11	16
1	1	0	13	20
1	1	1	15	10

Question 11: Simulate the krimp algorithm if we assume $F = \{AB, ABC, AC\}$ (already sorted as in krimp). You have to give the itemsets in the code table at each iteration of krimp (you do not need to give the items), the itemset considered for inclusion in the code table and if it is included and why.

5 Sequences (3 pts)

Sid	Sequence
1	(abc)abc
2	(abc)(abc)(ac)
3	ac(ac)bac
4	bac(abc)(ac)

Question 12: Of which sequences of the database are sequences $S1 = a(ab)$ and $S2 = (ac)ac$ subsequences? You have to give the correspondence between the items of $S1$ and $S2$ and the items of the sequences of the database. What are the support of $S1$ and $S2$ in this database?

Question 13: Given the following set of frequent sequences of size 3, what are the candidates sequences of size 4? How do you compute them? $F_3 = \{abc, b(ab), bac, bcb, c(ab)\}$